사내 코딩 어시스턴트 도입 방식 검토

벡터 RAG + 웹 UI 방식의 한계와 구조 기반 MCP 방식 제안

Pro*C / ProFrame 코드베이스에 최적화된 접근 방식

PoC 비교 검증 완료

2026.03

Contents

01 **현재 제안 검토**
벡터 RAG + 웹 UI 코딩 어시스턴트 방식의 구조

02 **Pro*C 환경에서의 한계**
벡터 RAG가 Pro*C 코드베이스에서 왜 작동하지 않는가

03 **대안: 구조 기반 MCP**
ProFrame의 구조 정보를 그대로 활용하는 접근

04 **PoC 비교 결과**
동일 코드베이스에서 두 방식 정량 비교

05 **구조 기반 방식의 실제 동작**
개발자 시나리오별 시연

06 **도입 방안 비교**
구현 복잡도, 비용, 유지보수 비교

07 **제안**
권장 접근 방식 및 다음 단계

01

현재 제안 검토

벡터 RAG + 별도 웹 UI 코딩 어시스턴트를 구축하는 방식

제안된 구조

1. 코드를 텍스트 청크로 분할 (6~50줄)
2. 임베딩 모델로 벡터화 → 벡터 DB 저장
3. 사용자 질문 → 유사도 검색 → 관련 청크 반환
4. LLM이 청크 기반으로 답변 생성
5. 별도 웹 UI에서 사용

필요 인프라:

- 임베딩 모델 (GPU 또는 API)
- 벡터 DB (ChromaDB / Pinecone 등)
- 웹 서버 + 프론트엔드
- LLM API 연동

이 방식의 전제 조건

코드가 자연어 텍스트처럼 작동한다고 가정

→ 유사한 단어 = 유사한 기능

이 전제가 맞는 경우:

- 일반 텍스트 문서 검색
- 다양한 언어/프레임워크의 오픈소스 코드
- 함수명/변수명이 자기 설명적인 코드

이 전제가 틀리는 경우:

- 같은 변수명이 다른 도메인에서 반복
- 비즈니스 로직이 SQL에 있는 코드
- 호출 관계가 코드 텍스트에 드러나지 않는 코드

질문: Pro*C/ProFrame 코드베이스가 이 전제에 맞는가?

02

Pro*C 환경에서 벡터 RAG의 한계

문제 1: 노이즈

tax_amt, acct_no, fee_rate,
HOLD_QTY, EVAL_AMT ...

이 변수명들이 주식, 채권, 펀드,
배당, 대출 등 모든 도메인에서
동일하게 사용됨.

→ 벡터 유사도가 높지만
실제로는 무관한 코드

문제 2: 컨텍스트 단절

Pro*C 함수를 이해하려면:
- 호출하는 다른 함수
- 입출력 구조체 (struct)
- 참조 테이블 DDL

이 모두 다른 파일에 있음.
청크 검색은 한 파일의 조각만
반환하므로 전체 그림이 안 보임.

문제 3: 구조 낭비

ProFrame이 이미 제공하는 것:
- 서비스 등록 정보 (이름, 설명)
- FDL (입출력 구조체 정의)
- EXECUTE_SERVICE (호출 관계)

벡터 RAG는 이 구조 정보를
전부 무시하고 코드를 텍스트
조각으로 취급함.

Pro*C/ProFrame 코드베이스의 특성

C + SQL 혼합 | 동일 변수명이 도메인 간 반복 | 비즈니스 로직 = EXEC SQL | ProFrame이 이미 구조화

**결론: 구조화된 코드에 비정형 텍스트 검색을 적용하는 것은
엑셀 테이블이 있는데 자연어로 검색하는 것과 같다.**

03

대안: 구조 기반 MCP + 기존 AI 코딩 도구 활용

Claude Code (기존 AI 코딩 도구)

svn-bridge MCP

SVN 이력 조회
커밋 로그, diff, blame
코드 읽기 (svn_cat)

proc-retrieval MCP

코드 구조 검색 (10개 Tool)
호출 그래프, 영향도, 업무 흐름
SQLite 경량 DB

Read / Edit (내장)

실제 코드 파일 읽기/수정
Claude Code 내장 기능
Repo 원본 직접 접근

핵심 차이점

- 별도 웹 UI 개발 불필요 → Claude Code가 UI
- 임베딩 모델/GPU 불필요 → SQLite 파일 1개
- 코드를 청크로 자르지 않음 → 구조 그대로 활용
- MCP는 경로만 반환 → 코드는 repo에서 직접 Read
- ProFrame 구조 정보를 100% 활용

MCP(Model Context Protocol)란?

AI 코딩 도구가 외부 데이터 소스에
접근할 수 있게 해주는 표준 프로토콜.

Claude Code, Cursor, Windsurf 등
주요 AI 코딩 도구가 지원.

→ 한 번 만들면 여러 도구에서 사용 가능

04

PoC 비교 결과

동일 코드베이스(128파일, 81함수)에서 10개 시나리오로 정량 비교

지표	벡터 RAG	구조 기반	의미
Recall (정답을 찾았는가)	40.0%	**98.0%**	구조 기반이 2.5배 높은 정답률
Noise (잘못된 결과 비율)	22.0%	**0.6%**	벡터는 노이즈가 5건 중 1건
Completeness (완전한 컨텍스트)	52.5%	**97.5%**	구조 기반은 호출함수+DDL까지 포함
Precision (반환 중 정답 비율)	34.0%	**40.7%**	구조 기반이 유틸 함수도 포함해서 비슷

왜 이런 차이가 나는가?

벡터 RAG:
"환매 수수료 계산" 검색 → tax_amt, fee_rate 등
유사 변수명을 가진 채권/배당/대출 코드가 노이즈로 반환

구조 기반:
"환매 수수료 계산" → RD_FEE_03 특정
→ 호출 그래프(CM_TAX_01, CM_DATE_CHK) + DDL 자동 확장

05

구조 기반 방식의 실제 동작

"이 함수 수정하면 영향 어디?"	get_impact("CM_TAX_01") → 9개 직접 호출자 + 9개 간접 호출자 depth별 영향 범위 즉시 파악
"이 테이블 누가 쓰지?"	find_by_table("TB_ACCT_MST") → 14개 컬럼, SELECT/INSERT/UPDATE별 사용 함수 목록
"이 업무 전체 흐름 보여줘"	get_business_flow("FD_PAY_01") → 호출 트리 + 각 단계 SQL 연산 (SELECT TB_FUND_DIST → FD_TAX_02 → ...)
"이 에러 어디서 나는 거야?"	find_by_error_code("E101") → ST_BUY_01 "주문가능금액 부족" 에러 설정 위치 + 주변 코드
"이 함수 코드 수정해줘"	get_function → Read → Edit → 구조 파악(MCP) → 코드 로드(Read) → 수정(Edit) — 항상 최신 코드

06

도입 방안 비교

항목	벡터 RAG + 웹 UI	구조 기반 MCP + Claude Code
UI 개발	웹 프론트엔드 + 백엔드 별도 구축 필요	불필요 Claude Code가 UI
인프라	임베딩 모델 (GPU/API) 벡터 DB 서버 웹 서버	SQLite 파일 1개 (수 MB)
코드 최신성	임베딩 재생성 필요 (코드 변경 시마다)	repo에서 직접 Read 항상 최신
검색 정확도	Recall 40% Noise 22%	Recall 98% Noise 0.6%
유지보수	임베딩 모델 업데이트 벡터 DB 관리 웹 서버 운영	SQLite 재인덱싱 (수 초)
구현 기간	프론트 + 백엔드 + RAG 파이프라인 = 수 개월	MCP 2개 + 파서 = 4주

07

제안

사내 코딩 어시스턴트 도입 시, 벡터 RAG + 별도 웹 UI 구축 대신

ProFrame 구조 정보 기반 MCP 서버를 구축하고, 기존 AI 코딩 도구(Claude Code 등)와 연동

필요 리소스

개발자 1명 (파서 + MCP 구축, 3주)

업무 시니어 1명 (서비스 설명 검토, 2~3일)

인프라:

- SQLite 파일 1개 (수 MB)
- SVN 접근 권한
- Claude Code 라이선스

구현 일정 (4주)

1주	Pro*C 파서 개발 실제 .pc 파일 파싱 검증
2주	MCP 서버 구축 proc-retrieval + svn-bridge
3주	전체 코드베이스 인덱싱 검색 정확도 튜닝
4주	Claude Code 연동 개발자 파일럿 테스트

다음 단계

1. 실제 Pro*C 파일 10개로 파서 검증
2. ProFrame 서비스 설명 품질 확인
3. 파일럿 팀 선정